



OASIS Security Report

Cross-Site Request Forgery Security Analysis

Date: 2026-06-11 09:59:17 — **Model:** mistral:latest

Vulnerability: Cross-Site Request Forgery

Project: clean_contracts

Analysis root: `../clean`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

Analyzed **2** file(s) for Cross-Site Request Forgery.

[Return to table of contents](#)

Statistics

Files analyzed: 2 **Total findings:** 2

Critical

(

0

High		0
Medium		2
Low		0

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/clean/modifier_reentrancy_fixed.sol</code>	0.637
<code>/work/project/dataset/clean/simple_dao_fixed.sol</code>	0.629

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.637

Source lines: 1-42

► Finding 1.1: Vulnerability found (Medium)

Vulnerable code

```
function airDrop() supportsToken hasNoBalance public{
```

Explanation

The function allows a user to call the airDrop function without checking if it is a Cross-Site Request Forgery request.

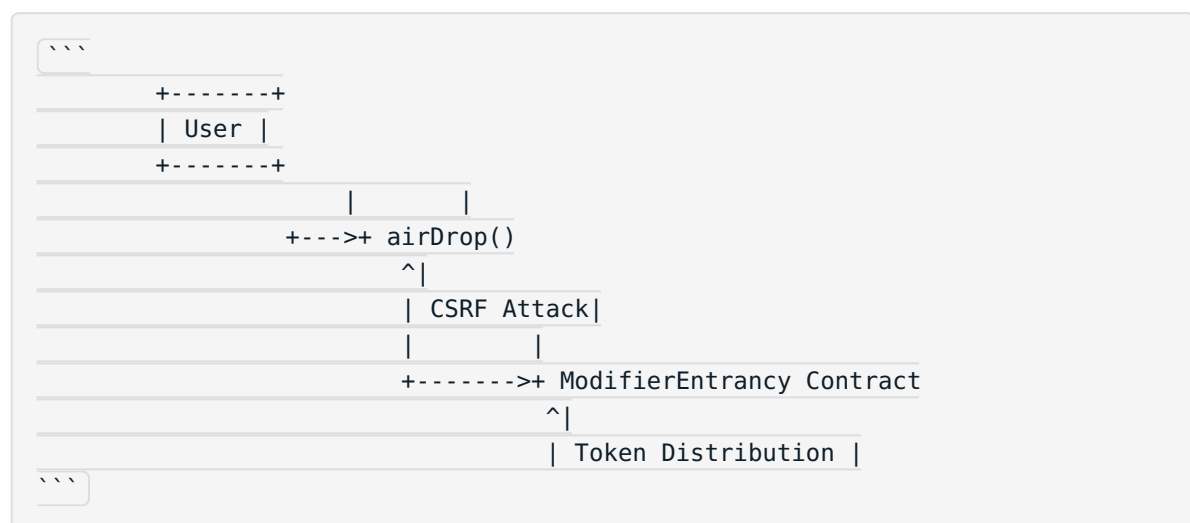
Impact

An attacker could potentially manipulate token distribution by making unauthorized requests.

Entry point

airDrop() function

Execution path



HTTP methods

Exploitation steps

- An attacker creates a malicious website.
- The user visits the attacker's website.

- The attacker uses CSRF to call the `airDrop()` function on behalf of the user.

Conditions

An attacker can trick the user into making a request to the `airDrop()` function.

Remediation

Implement CSRF protection by adding a CSRF token or using secure methods for form submission.

Secure example

```
function airDrop(csrfToken) supportsToken hasNoBalance public {  
    require(msg.send(csrfToken) == expectedCsrftoken, 'Invalid CSRF token');  
    tokenBalance[msg.sender] += 20;  
}
```

File 2: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.629

Source lines: 1-34

► Finding 2.1: Vulnerability found (Medium)

Vulnerable code

```
function withdraw(uint amount) public {  
    if (credit[msg.sender]>= amount) {  
        credit[msg.sender]-=amount;  
        require(msg.sender.call.value(amount)());  
    }  
}
```

Explanation

This function allows state change based on the user's address, which can be manipulated via a malicious website through a CSRF attack.

Impact

An authenticated user's credit balance may be reduced unintentionally.

Entry point

withdraw function

Execution path

```
***  
user -> malicious site -> smart contract - withdraw(amount)  
***
```

Parameters

address to

Exploitation steps

- User visits malicious website with embedded form.
- Form submits CSRF request to the contract's withdraw function.

Conditions

Attacker can reach the affected function via a web-based request.

Remediation

Use CSRF tokens or SameSite cookies for form submissions to prevent CSRF attacks.

Secure example

```
// Example with CSRF token
function withdraw(uint amount, bytes memory _csrfToken) public {
    require(_verifyCsrfToken(_csrfToken));
    if (credit[msg.sender]>= amount) {
        credit[msg.sender]-=amount;
        require(msg.sender.call.value(amount)());
    }
}
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)